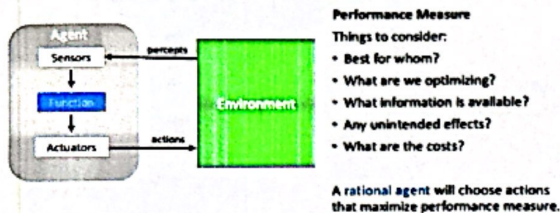


What is AI:

1. The study of agents that receive percepts from the environment and perform actions.
2. The science and engineering of making intelligent machines.

Intelligent Agents:

Performance Measure, Environment, Actuators, Sensors



Properties of Task Environment:

- Fully Observable (vs. Partially Observable):

An agent's sensors give it access to the complete state of the environment at each point of time.

- Single agent (vs. Multi-agent):

An agent operating by itself in an environment.

- Deterministic (vs. Stochastic):

The next state of the environment is completely determined by the current state and the action executed by the agent.

- Strategic:

If the environment is also dependent on the actions of other agents, then it is also strategic. However, if other agents are predictable, i.e., their strategies are fixed and only dependent on the state, then the other agents can be subsumed as parts of the environment.

- Episodic (vs. Sequential):

The agent's experience is divided into atomic "episodes", and the choice of action in each episode depends only on the episode itself.

- Static (vs. Dynamic):

The environment is unchanged while an agent is deliberating. The environment is semi-dynamic if the environment itself does not change with the passage of time, but the agent's performance score does.

- Discrete (vs. Continuous):

A limited number of distinct, clearly defined percepts and actions.

Agent structures:

- Simple reflex agent: fixed conditional rules.
- Goal-based agent: Keep in mind of a goal, and action aims to achieve it.
- Utility-based agent: Choose the steps that has highest utility score.
- Learning agent: Improve its performance over time by using experience.

Intelligent Agents: Exploration vs **Exploitation**

Exploration: Learn more about the world

Exploitation: Maximize gain based on current knowledge

Problem Formulation in (Systematic) Search:

- **States**: Each state corresponds to an **actual/specific situation or configuration** of the problem domain.
 - Representation invariant: ensure that the abstract states have corresponding concrete states.
- **Initial State**: The starting configuration
- **Goal Test / State(s)**:
 - A function that checks whether a state is the goal state, or
 - A collection of goal state(s)
- **Actions**
 - For every state, we have a set of actions, denoted by $actions(state)$ with size $|actions(state)|$.
 - An integer b is called a branching factor, if for all states $|actions(state)| \leq b$.
- **Transition model**
 - Given a state and action, the transition function produces the next state
 - $new_state = transition(state, action)$
- **Action cost function**: map state and action to a cost

Completeness and Optimality:

- **Completeness**: For every problem instance, it will find the solution if one exists.
- **Optimality**: For every instance it produces a solution, that solution is the best possible.

Uniform Search Algorithms (AKA Blind Search):

- We don't know how good a state is, i.e., we can't know any information of how close a state is to the goal.
- We only know the **problem formulation**, and in particular the **action cost** (step cost).

Breadth-first Search (BFS)

- Time complexity: (#nodes generated) **Exponential** w.r.t. the depth of the optimal solution.
- Space complexity: (size of the frontier **Queue**) **Exponential** w.r.t. the depth of the optimal solution.
- Completeness: Yes, if the problem has a **finite branching factor** and there exists a goal at a finite depth.
- Optimality: Yes, if all actions have equal positive cost.

Uniform-cost Search (UCS)

- Time complexity: (#nodes generated) **Exponential** w.r.t. the "tier" of the optimal solution.
- Space complexity: (size of the frontier **Priority Queue**) **Exponential** w.r.t. the "tier" of the optimal solution.
- Completeness: Yes, if the search tree has a **finite branching factor**, **every action cost $\geq \epsilon$** , and a goal is reachable with finite optimal path cost that is **bounded**.
- Optimality: Yes, if all costs are nonnegative.

Depth-first Search (DFS)

- Time complexity: (#nodes generated) **Exponential** w.r.t. the maximum depth of the search tree.
- Space complexity: (size of the frontier **Stack**) Polynomial w.r.t. the maximum depth of the search tree.
- Completeness: **No**, the depth of search tree could be infinite. Yes, if with visited memory.
- Optimality: No, the optimal solution might be in a shallower path.

Informed Search Algorithms (AKA Search with extra info):

- The algorithm has some clue how good a state is, i.e., how close a state is to the goal.
- This knowledge is due to the use of a heuristic function.

A heuristic $h(n)$ is an estimate of the optimal path cost from any state to the goal state.

- All heuristic functions must be **non-negative** and have $h(goal) = 0$.
- A heuristic $h(n)$ is **admissible** if for every node n , $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the optimal path cost to reach the goal state from n .
- A heuristic $h(n)$ is **consistent** if for every node n , every successor n' of n generated by any action a , $h(n) \leq c(n, a, n') + h(n')$. **直线更短**
- If $h_1(n) \geq h_2(n)$ for all n , then h_1 dominates h_2 . i.e., h_1 is better for search if admissible.

A* Search

- Idea: UCS on $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach n , and $h(n)$ is heuristic.
- Time complexity: (#nodes generated) **Exponential**, good heuristic can improve
- Space complexity: (size of the frontier **Priority Queue**) **Exponential**
- Completeness: Yes, if satisfies requirements for UCS and additional for A*.

Consistency $\longrightarrow$ **A* with or without memory optimal and complete**

Admissibility $\longrightarrow$ **A* without memory optimal and complete**

Depth-limited Search (DLS)

- Idea: Limit the search to depth l . Backtrack when limit is hit.
- Time complexity: (#nodes generated) **Exponential** w.r.t. the depth limit
- Space complexity: (size of the frontier) Polynomial w.r.t. the depth limit if used with DFS.
- Completeness: **No**
- Optimality: **No**, if used with DFS

Iterative Deepening Search (IDS)

- Idea: Search with depth limit $0, 1, \dots, \infty$. Return solution if found.
- Time complexity: (#nodes generated) **Exponential** w.r.t. the depth of optimal solution (with overhead)
- Space complexity: (size of the frontier) Polynomial if used with DFS.
- Completeness: Yes, under the same conditions that guarantee BFS completeness.
- Optimality: Yes, under the same conditions that guarantee BFS optimality.

Name	Time Complexity ¹	Space Complexity ²	Complete?	Optimal?
Breadth-first Search	Exponential	Exponential	Yes	Yes
Uniform-cost Search	Exponential	Exponential	Yes ³	Yes ³
Depth-first Search	Exponential	Polynomial ²	No Yes ⁴	No
Depth-limited Search	Exponential	Polynomial ²	No	No ²
Iterative Deepening Search	Exponential	Polynomial ²	Yes	Yes
A* Search	Exponential	Exponential	Yes ³	Yes ³ if admissible
A* Search with visited memory	Exponential	Exponential	Yes ³	Yes ³ if consistent

- 对 unit cost 最优
- 1) In terms of some notion of depth/tier
 - 2) If used with DFS
 - 3) If edge costs are positive
 - 4) With visited memory

A IDS 就是用 DFS 做 DLS, 但深度上限直接上到无穷, 直到找到解为止。

① 问题本身
(1) 有解, 则问题有解
(2) goal == M, 唯一终态
(但如果 goal state 是满足某个性质的, 则可能多 goal states)
(iii) 有解, 若动作集中有 "能单步移动到任意状态" 的动作, 则有解。
② tree 是否无限?
(1) 理论上无限 + cycles
→ 无限。
(ii) terminate?
DFS/UCS 存在解且耗时 70%
complete
DFS: 不保证

DLS 是穷举 ① 分支因子有限 ② 解一定存在深度 $\leq l$ 的树
最优: (树搜索) 若说 "解唯一" 则 DLS 最优

Local Search:

- Simple strategy: Traverse the state space by considering only the best **locally** reachable state.
- Typically, **incomplete** and **suboptimal**.
- Any-time property: longer run time, (often) better solution.

Problem formulation in Local Search:

- States:** Represent different configurations or candidate solutions. They may or may not map to an **actual problem state** but are used to represent potential solutions.
- Initial State:** The starting configuration (candidate solution).
- Goal Test (optional):** Check if a current state is the desired solution.
- Successor Function:** A function that generates neighboring states (candidate solutions) by applying modifications to the current state.

Evaluation function:

- A mathematical function used to assess the quality or desirability of a state.
- The function takes in a state as an argument and produces a number. $value = eval(state)$

Hill climbing algorithm:

current_state = initial_state

while True:

best_successor = a **highest-eval** successor state of **current_state**

if **eval(best_successor) <= eval(current_state):**

return **current_state**

current_state = best_successor

Problem formulation in Adversarial Search:

- States**
- Initial State**
- Terminal States:** states where the game ends, such as win/lose/draw states.
 - This set of states could be defined as a function.
- Actions**
- Transition**
- Utility Function:** output the value of a state from the perspective of our agent.
 - Our agent (player A) wants to **maximize** the utility.
 - In games such as Chess or tic-tac-toe, the outcome for player A is a win, loss, or draw.
 - Assign utility values of +1, -1, and 0, respectively, to represent these outcomes.
 - In other games such as Go with territory scoring, the outcome for player A is some number.
 - Assign integer/real valued utility values to the outcomes.
 - The utility of non-terminal states is typically not known in advance and must be computed.

Minimax - Analysis

```
def minimax(state):
    (action, v) = max_value(state)
    return action

def max_value(state):
    if is_terminal(state): return utility(state)
    v = -inf // smallest value possible
    for next_state in expand(state):
        v = max(v, min_value(next_state))
    return v
```

```
def min_value(state):
    if is_terminal(state): return utility(state)
    v = inf // largest value possible
    for next_state in expand(state):
        v = min(v, max_value(next_state))
    return v
```

Time Complexity?

Exponential w.r.t. maximum depth: $O(b^M)$
Branching factor b , max-depth m

Space Complexity?

Polynomial

Complete?

Yes, if tree is finite

Optimal?

Yes, against optimal opponent

Alpha-Beta Pruning

```
def alpha_beta_search(state):
    (action, v) = max_value(state, -inf, inf)
    return action

def max_value(state, alpha, beta):
    if is_terminal(state): return utility(state)
    v = -inf
    for next_state in expand(state):
        v = max(v, min_value(next_state, alpha, beta))
        alpha = max(alpha, v)
        if v >= beta: return v
    return v

def min_value(state, alpha, beta):
    if is_terminal(state): return utility(state)
    v = inf
    for next_state in expand(state):
        v = min(v, max_value(next_state, alpha, beta))
        beta = min(beta, v)
        if v <= alpha: return v
    return v
```

Alpha-Beta Pruning does not alter the minimax value, and the optimal move computed for the root node. It only reduces the number of nodes evaluated.

Cutoff:

Halting the search in the middle and estimate the value of states using an evaluation function.

- Evaluation function:
- 1. If terminal state: use utility(state)
- 2. Otherwise: use a heuristic

- Heuristic function:

A function that estimates the utility of the state. It should fall somewhere between the minimum and maximum utility.

- The minimax algorithm with cutoff and an evaluation function returns a move that is **optimal with the respect to the evaluation function at the cutoff**.
- This move may be **suboptimal** with respect to the true minimax value

Machine Learning:

- Supervised Learning:**
Learn from labeled data (input-output pairs) to learn a mapping from inputs to outputs.
- Unsupervised Learning:**
Learn from unlabeled data (input only) to find patterns or structure.

Confusion Matrix

		True Label	
		Cancer	Benign
Predicted Label	Cancer	2 True Positive (TP)	1 False Positive (FP)
	Benign	3 False Negative (FN)	4 True Negative (TN)

$$Accuracy = \frac{TP+TN}{TP+FP+FN+TN}$$

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

$$F1 \text{ Score} = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}}$$

Background: Entropy for Binary Outcomes

For a dataset which contains p positive and n negative outcomes.

Probabilities:

$$P(pos) = \frac{p}{p+n} \text{ and } P(neg) = \frac{n}{p+n}$$

Entropy:

$$H(p) = -H(P(pos), P(neg)) = -\sum_{x \in \{pos, neg\}} P(x) \log_2 P(x)$$

Base cases:

- If no data is available: use default.
- If same decision for all rows: use it.
- If attributes are empty: use majority.

Prunings:

- Max depth: limits the maximum depth of the decision tree. The root node has a depth of 0.
- Min-sample leaves: set a minimum threshold for the number of samples required to be in a leaf node.

$$p = \sigma(z) = \frac{1}{1+e^{-z}} \quad p \geq 0.5 \Leftrightarrow z \geq 0$$

$$h_w(x) = \sigma(w_0 x_0 + w_1 x_1 + w_2 x_2 + \dots + w_d x_d)$$

$$\sigma(x) = \frac{1}{1+e^{-x}} \quad \sigma'(x) = \sigma(x)(1-\sigma(x)) \quad h_w(x) = \sigma(w^T x) = \sigma\left(\sum_{j=0}^d w_j x_j\right)$$

Decision Threshold: a probability set to decide whether an input belongs to a certain class.

Decision boundary: a surface that separates the different classes in the feature space. It is an intersection between the model's function and the decision threshold.

Binary Cross-Entropy (BCE):

$$BCE(y, p) = -y \log(p) - (1-y) \log(1-p) \quad y = 1 \text{ or } 0$$

$$J_{BCE}(w) = \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, h_w(x^{(i)})) \quad \frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, h_w(x^{(i)}))$$

$$w_j \leftarrow w_j - \gamma \frac{\partial J_{BCE}(w_0, w_1, \dots)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Binary Classifier	One-vs-One	One-vs-Rest
For every pair of classes.	For each class, treating all other classes as a single combined class.	
Number of binary classifiers	$C(K, 2) = \frac{K(K-1)}{2}$	K
Prediction	Each classifier votes for a class, and the class with the most votes is selected.	The classifier with the highest probability output (confidence score) determines the class.
Example	Classifier 1: Distinguish between class 1 and 2. Classifier 2: Distinguish between class 1 and 3. Classifier 3: Distinguish between class 2 and 3.	Classifier 1: Distinguish class 1 and not-1 (2, 3). Classifier 2: Distinguish class 2 and not-2 (1, 3). Classifier 3: Distinguish class 3 and not-3 (1, 2).

Multi-label: Common Techniques

- Binary Relevance: Train a separate model for each label.
- Classifier Chain: Like Binary Relevance but feed predictions of previous labels as features for subsequent labels.
- Label Powerset: Treat each unique label-set as a single multiclass label.

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1, \dots)}{\partial w_j}$$

$$w = (X^T X)^{-1} X^T Y$$

	Gradient Descent	Normal Equation
Need to choose learning rate γ	Yes	No
Iteration(s)	Many	None, closed-form solution
Large number of features d ?	No problem	Slow, $(X^T X)^{-1} \rightarrow O(d^3)$
Feature scaling?	May be necessary	Not necessary
Constraints	-	$X^T X$ needs to be invertible

Varlants of Gradient Descent:

- (Batch) Gradient Descent: Consider all training examples.
- Mini-Batch Gradient Descent: Consider a subset of training examples at a time. Noisy gradient may help escape local minima.
- Stochastic Gradient Descent: Select one random training example at a time. Very Noisy gradient may help escape local minima.

Solutions for Features with Different Scales

Scale features to be within a common range. Also known as **normalization**.

Min-max scaling

$$x_i' = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

Scales the features to be within [0,1].

It is also common to scale the features to be within [-1,1].

Standardization

$$x_i' = \frac{x_i - \mu_i}{\sigma_i}$$

This method transforms features to have a mean of 0 and a standard deviation of 1.

Other solution: Different learning rate γ_i for each weight.

Dataset Quality:

- Relevance: Dataset should contain relevant data.
- Noise: Dataset may contain noise (irrelevant or incorrect data).
- Balance (for classification):
Balanced datasets ensure that all classes are adequately represented.

$$J_{MSE}(w) = \frac{1}{2} (h_w(x^{(i)}) - y^{(i)})^2$$

$$\frac{\partial J_{MSE}}{\partial w_0} = (h_w(x^{(i)}) - y^{(i)})$$

$$\frac{\partial J_{MSE}}{\partial w_1} = (h_w(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

$$e_i = h_w(x^{(i)}) - y^{(i)}$$